

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	10% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	Ramya S S	Reg. No.:	192421226
Section / Batch:	B.Tech IT	Date:	22.08.2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Analyze the given questions thoroughly before answering.
- Compare multiple aspects such as methods, results, advantages, and limitations clearly.
- Critically evaluate the strengths, weaknesses, and implications with proper justification.
- Present meaningful insights, interpretations, and well-supported conclusions from the comparisons.

Question Type	Focus Area	Questions
Comparative Analysis	Comparative analysis of Dynamic Programming and Greedy algorithms for optimization problems.	Q1

SECTION A – Comparative Analysis

Code of Conduct

I Ramya S S/192421226 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Ramya S S

Comparative Analysis of Longest Palindromic Subsequence and Word Break Using Dynamic Programming

1. Introduction

Dynamic Programming (DP) is an algorithmic technique that solves complex problems by dividing them into smaller overlapping subproblems and storing their results. This assessment compares two important string-based DP problems: **Longest Palindromic Subsequence (LPS)** and **Word Break**. LPS finds the longest subsequence that forms a palindrome, while Word Break checks whether a string can be divided into valid words from a given dictionary. The comparison focuses on their algorithms, state transitions, table structures, complexity, correctness, and scalability.

2. Longest Palindromic Subsequence (LPS)

Problem Definition

Given a string S , find the longest subsequence of S that reads the same forward and backward.

Example:

Input: BBABCBCAB

Output: 7

One longest palindromic subsequence is BABCBAB.

Algorithm Steps

1. Let n be the length of the string.
2. Create a 2D DP table $dp[n][n]$.
3. Set $dp[i][i] = 1$ because every single character is a palindrome.
4. Consider substrings in increasing order of length.
5. If $S[i] == S[j]$, use:
$$dp[i][j] = dp[i+1][j-1] + 2$$
6. If $S[i] != S[j]$, use:
$$dp[i][j] = \max(dp[i+1][j], dp[i][j-1])$$
7. The final answer is $dp[0][n-1]$.

Pseudocode

LPS(S):

$n = \text{length}(S)$

create $dp[n][n]$

for $i = 0$ to $n-1$:

$dp[i][i] = 1$

```

for length = 2 to n:
    for i = 0 to n-length:
        j = i + length - 1

        if S[i] == S[j]:
            dp[i][j] = dp[i+1][j-1] + 2
        else:
            dp[i][j] = max(dp[i+1][j], dp[i][j-1])

return dp[0][n-1]

```

Correctness

If the two boundary characters are equal, they can form part of the palindrome with the optimal inner subsequence. If they are different, the algorithm considers excluding either boundary character and selects the larger result.

Complexity

- **Time Complexity:** $O(n^2)$
- **Space Complexity:** $O(n^2)$

3. Word Break

Problem Definition

Given a string and a dictionary of valid words, determine whether the complete string can be divided into one or more dictionary words.

Example:

String: leetcode

Dictionary: ["leet", "code"]

Output: True

Segmentation: leet + code

Algorithm Steps

1. Let n be the length of the string.
2. Create a 1D DP array $dp[n+1]$.
3. Set $dp[0] = \text{True}$, representing an empty valid prefix.
4. For each position i , check all previous positions j .
5. If $dp[j]$ is true and $S[j:i]$ is present in the dictionary, set $dp[i] = \text{True}$.
6. Continue until the entire string is processed.
7. The final answer is $dp[n]$.

Pseudocode

WordBreak(S, Dictionary):

n = length(S)

create dp[n+1]

dp[0] = True

for i = 1 to n:

for j = 0 to i-1:

if dp[j] == True and S[j:i] in Dictionary:

dp[i] = True

break

return dp[n]

Correctness

If a prefix ending at position j is valid and the substring from j to i is a dictionary word, then the prefix ending at i is also valid. Therefore, dp[n] correctly determines whether the complete string can be segmented.

Complexity

- **Time Complexity:** $O(n^2)$ for the basic DP approach.
- **Space Complexity:** $O(n)$, excluding dictionary storage.

4. Comparison of DP Table Structures

Feature	Longest Palindromic Subsequence	Word Break
Objective	Find longest palindromic subsequence	Check valid word segmentation
DP Structure	2D table	1D array
State	dp[i][j]	dp[i]
Represents	Best palindrome in substring i to j	Whether prefix of length i is valid
Transition	Based on character comparison	Based on dictionary words
Output	Maximum length	True/False
Time	$O(n^2)$	$O(n^2)$
Space	$O(n^2)$	$O(n)$

5. State Transition Comparison

LPS Transition

If the boundary characters are equal:

$$dp[i][j] = dp[i+1][j-1] + 2$$

Otherwise:

$$dp[i][j] = \max(dp[i+1][j], dp[i][j-1])$$

LPS therefore depends on smaller substrings within the current range.

Word Break Transition

If $dp[j]$ is true and $S[j:i]$ is a valid dictionary word:

$$dp[i] = \text{True}$$

Word Break therefore progresses from left to right using previously solved prefixes.

6. Difference in Problem Objectives

LPS is an **optimization problem** because it attempts to maximize the length of the palindrome. Several possible subsequences may need to be compared to find the best result.

Word Break is a **decision problem** because it only needs to determine whether at least one valid segmentation exists. Once a valid prefix is found, its result can be reused for later positions.

7. Scalability Analysis

LPS requires a 2D table, resulting in $O(n^2)$ memory usage, which can become expensive for very large strings. Word Break requires only a 1D DP array, so it is more memory-efficient and generally scales better in terms of space. Efficient dictionary structures such as hash sets or tries can further improve Word Break performance. Thus, the choice of DP representation plays an important role when handling large input strings.

8. Key Insights on String-Based DP

1. **State representation matters:** LPS requires two indices, while Word Break mainly requires one index.
2. **Different objectives use different transitions:** LPS maximizes a value, whereas Word Break checks a Boolean condition.
3. **DP avoids repeated computation:** Previously calculated substring or prefix results are stored and reused.
4. **Space complexity affects scalability:** LPS uses $O(n^2)$ space, while Word Break generally uses $O(n)$ space.

9. Conclusion

LPS and Word Break demonstrate how Dynamic Programming can be applied to different string-based problems. LPS uses a 2D table to find the maximum palindromic subsequence length, while Word Break uses a 1D table to determine valid word segmentation. Although both have a typical time complexity of $O(n^2)$, Word Break is more space-efficient because it requires only $O(n)$ additional DP space. The comparison shows that selecting the correct DP state and transition is essential for solving string problems efficiently and achieving better scalability.

RUBRICS FOR CALCULATION

Comparative Analysis					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Selection of Studies/Parameters	20%	Selects highly relevant and diverse studies with well-defined comparison parameters	Selects relevant studies with minor gaps in parameter definition	Limited or partially relevant selection	Poor or inappropriate selection of studies
Comparison & Differentiation	25%	Clearly compares multiple aspects (methods, results, limitations) with strong differentiation	Good comparison with minor gaps	Basic comparison with limited depth	No clear comparison; mostly descriptive
Critical Evaluation	25%	Critically evaluates strengths, weaknesses, and implications with strong justification	Provides evaluation with some justification	Limited evaluation; lacks depth	No critical evaluation provided
Synthesis & Insight Generation	20%	Generates meaningful insights and conclusions from comparisons	Some insights derived from comparison	Limited or unclear insights	No insights generated
Clarity	10%	Information is clearly presented (tables/figures) with logical structure	Generally clear with minor issues	Presentation lacks clarity or structure	Poor presentation and difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Selection of Studies/Parameters	20%				
Comparison & Differentiation	25%				
Critical Evaluation	25%				
Synthesis & Insight Generation	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____